MESTRADO PROFISSIONAL EM COMPUTAÇÃO APLICADA

Estruturas de Dados e Análise de Algoritmos

P1 Resolvida

Plano de Reflorestamento Sustentável com Drones

Material de estudo/preparação para provas futuras, a partir de uma prova (P1) já aplicada em turma anterior, com apoio de IA (Claude). Cada resposta traz primeiro o raciocínio aritmético em linguagem simples e, em seguida, um programa Java real (executado nesta sessão, Java 21) que confirma numericamente o resultado — não são números estimados.

Algoritmo da prova, em resumo

PlanoReflorestamentoSustentavel(A, nivelCritico) faz 5 coisas em sequência: (1) ordena o vetor A com Selection Sort; (2) soma um custo que dobra a cada posição (peso = 2^i); (3) faz busca binária por nivelCritico; (4) planeja missões de drone dividindo o vetor ao meio recursivamente, inspecionando tudo a cada nível; (5) roda um laço de monitoramento cujo laço interno tem tamanho logarítmico.

Questão 1 — Quantas comparações o Selection Sort faz?

Raciocínio simples: no Selection Sort, para achar o menor elemento restante, o primeiro passo compara o 1º elemento com os outros n-1; o segundo passo compara o 2º elemento restante com os outros n-2; e assim por diante, até restar só 1 elemento (0 comparações). Somando tudo:

```
(n-1) + (n-2) + (n-3) + ... + 1 + 0
```

Essa soma é a mesma soma que Gauss calculou quando criança: some o maior com o menor, o segundo maior com o segundo menor, etc. — cada par soma (n-1), e há n/2 pares, então o total é:

```
n(n-1)/2
```

Prova simples por indução: se a fórmula vale para n elementos, para n+1 elementos basta somar mais n comparações (o novo elemento comparado com todos os outros n já existentes):

```
n(n-1)/2 + n = n(n+1)/2  <- exatamente a fórmula com n+1 no lugar de n
```

Verificação em Java (comparações contadas de verdade vs. fórmula)

```
static long comparacoesSelectionSort(double[] a) {
    long comparacoes = 0;
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            comparacoes++;
            if (a[j] < a[min]) min = j;
        }
        double aux = a[i]; a[i] = a[min]; a[min] = aux;
    }
    return comparacoes;
}
```

```
n       comparacoes reais    formula n(n-1)/2    iguais?
10      45                    45                  true
100     4950                  4950                true
1000    499500                499500              true
5000    12497500              12497500            true
```

Para milhões de registros: n(n-1)/2 cresce muito mais rápido que n — dobrar n quadruplica o número de comparações. Com 1 milhão de áreas, seriam ~5×10^11 comparações: horas de processamento. Por isso Selection Sort não deve ser usado em uma base real de sensoriamento florestal em escala nacional.

Questão 2 — Soma dos pesos 2^i

Raciocínio simples: somar 1+2+4+8+...+2^(n-1) é como juntar moedas que dobram de valor a cada vez. O truque: se você soma todas as moedas e ainda soma mais uma vez a maior delas, o total vira exatamente o dobro da maior moeda. Por isso a soma de 2^0 até 2^(n-1) é sempre **uma unidade a menos** que a próxima potência de 2:

```
2^0 + 2^1 + ... + 2^(n-1) = 2^n - 1
```

Prova por indução, em português simples: vale para n=1 (2^0 = 1 = 2^1 - 1). Se vale para n, para n+1 basta somar mais um termo (2^n): (2^n - 1) + 2^n = 2·2^n - 1 = 2^(n+1) - 1 — a mesma fórmula, com n+1.

Verificação em Java (soma pelo laço vs. fórmula fechada)

```
static long somaPesosLoop(int n) {
    long soma = 0;
    for (int i = 0; i <= n - 1; i++) soma += (1L << i);
    return soma;
}
```

```
n    soma pelo loop   formula 2^n - 1   iguais?
1    1                1                 true
5    31               31                true
10   1023             1023              true
15   32767            32767             true
20   1048575          1048575           true
```

Por que isso preocupa em sustentabilidade: o último peso (2^(n-1)) sozinho já é maior que a soma de todos os anteriores. Ou seja, um único estágio final concentra quase todo o custo/energia do algoritmo — um jeito arriscado de distribuir custo. Pesos lineares (peso=i) ou constantes (peso=1) distribuiriam melhor.

Questão 3 — Por que a busca binária funciona (invariante)

Ideia simples: a busca binária sempre olha o meio do intervalo restante. Se o valor do meio for maior que o que procuro, descarto a metade direita (todos são ainda maiores); se for menor, descarto a metade esquerda. A cada passo, **o intervalo cai pela metade** — e o alvo, se existir, nunca é descartado por engano, porque cada descarte só remove valores que já provamos não poderem ser o alvo.

- **Início:** o intervalo é o vetor inteiro — obviamente contém o alvo, se ele existir.
- **Cada passo:** o intervalo some pela metade, mas continua garantindo conter o alvo (se existir).
- **Fim:** ou encontrou o alvo, ou o intervalo virou vazio — e aí, com certeza, o alvo não está no vetor.

Por que sempre termina: o tamanho do intervalo é um número que só diminui a cada passo (nunca fica igual) — uma sequência de números que só diminui não pode continuar para sempre, então o laço acaba em no máximo log2(n) + 1 passos.

Verificação em Java (contando as iterações reais)

```
static int buscaBinaria(int[] a, int alvo) {
    int inicio = 0, fim = a.length - 1;
    iteracoes = 0;
    while (inicio <= fim) {
        iteracoes++;
        int meio = (inicio + fim) / 2;
        if (a[meio] == alvo) return meio;
        else if (a[meio] < alvo) inicio = meio + 1;
        else fim = meio - 1;
    }
    return -1;
}
```

```
n = 1000000
alvo=0       -> posicao=0,      iteracoes=19, limite teorico ceil(log2 n)+1=21, dentro do limite? true
alvo=999999  -> posicao=999999, iteracoes=20, limite teorico ceil(log2 n)+1=21, dentro do limite? true
alvo=500000  -> posicao=500000, iteracoes=19, limite teorico ceil(log2 n)+1=21, dentro do limite? true
alvo=123456  -> posicao=123456, iteracoes=16, limite teorico ceil(log2 n)+1=21, dentro do limite? true
alvo=-1      -> posicao=-1,     iteracoes=19, limite teorico ceil(log2 n)+1=21, dentro do limite? true
```

Note que, mesmo em um vetor de 1 milhão de posições, nunca foram necessárias mais que 20 iterações — exatamente o comportamento logarítmico previsto.

Questão 4 — Quanto custa o planejamento recursivo?

Ideia simples: a função Planejar divide a lista de áreas ao meio e chama a si mesma para cada metade — só que, antes de dividir, ela inspeciona todas as áreas daquele pedaço. Então em cada "nível" da recursão (nível = uma rodada de divisões), o total de área inspecionada é sempre **n**, porque as metades juntas cobrem tudo. E quantos níveis existem? A cada nível o pedaço fica na metade do tamanho — então em log2(n) níveis o pedaço vira do tamanho 1 e a recursão para.

```
custo total = (custo por nível) x (numero de niveis) = n x log2(n)
```

Isso é exatamente T(n) = 2T(n/2) + n resolvido pelo Teorema Mestre, que dá Θ(n log n).

Verificação em Java (contando as inspeções reais)

```
static void planejar(int inicio, int fim) {
    if (inicio > fim) return;
    for (int i = inicio; i <= fim; i++) inspecoes++;
    if (inicio == fim) return;
    int meio = (inicio + fim) / 2;
    planejar(inicio, meio);
    planejar(meio + 1, fim);
}
```

```
n      inspecoes reais   n*log2(n) aprox   razao (deve ficar estavel)
8      32                 24.0              1.333
16     80                 64.0              1.250
64     448                384.0             1.167
256    2304               2048.0            1.125
1024   11264              10240.0           1.100
4096   53248              49152.0           1.083
```

A razão entre o valor real e n·log2(n) se estabiliza perto de 1 conforme n cresce — confirmando que o crescimento real acompanha n log n, como previsto pelo Teorema Mestre (a pequena diferença vem da constante extra pela forma como esse algoritmo específico soma o trabalho por nível).

Questão 5 — Laço de monitoramento escalonado

Ideia simples: para cada posição i, o laço interno divide i por 2 repetidamente até chegar a zero — ou seja, conta quantas vezes dá para dividir i por 2, que é justamente piso(log2 i) + 1 vezes (o mesmo processo de contar em quantos "cortes ao meio" um número de tamanho i se reduz a 1).

Somando essa contagem para todo i de 1 até n, a maioria dos valores de i está "perto" de n, e cada um deles contribui com algo perto de log2(n) — então a soma total fica perto de n·log2(n).

Verificação em Java (contagem real vs. n log2 n)

```
static long totalIteracoesEscalonado(int n) {
    long total = 0;
    for (int i = 1; i <= n; i++) {
        int nivel = i;
        while (nivel >= 1) { total++; nivel /= 2; }
    }
    return total;
}
```

```
n         total real   n*log2(n) aprox   razao
100       580          664.4             0.873
1000      8987         9965.8            0.902
10000     123631       132877.1          0.930
100000    1568946      1660964.0         0.945
```

A razão se aproxima de 1 conforme n cresce (0,87 → 0,95), confirmando $\Theta(n \log n)$. Comparando com n^2 : para n=100.000, n^2 seria 10.000.000.000 — mais de 6.000 vezes maior que o valor real medido (1.568.946), mostrando a enorme distância entre as duas classes de crescimento.

Questão 6 — Complexidade total e troca do algoritmo

Juntando o custo de cada etapa:

Etapa	Complexidade
1. Selection Sort	$\Theta(n^2)$
2. Soma geométrica	$\Theta(n)$
3. Busca binária	$\Theta(\log n)$
4. Planejamento recursivo	$\Theta(n \log n)$
5. Monitoramento escalonado	$\Theta(n \log n)$

Regra simples: quando somamos várias etapas, só a etapa mais "pesada" importa para n grande — as outras ficam pequenas demais para fazer diferença. Aqui, $\Theta(n^2)$ do Selection Sort é a mais pesada, então o algoritmo inteiro é $\Theta(n^2)$, mesmo as outras 4 etapas sendo mais eficientes.

Solução: trocar o Selection Sort por um algoritmo $\Theta(n \log n)$ (Merge Sort, ou simplesmente Arrays.sort/Collections.sort do Java). Depois da troca, a etapa mais pesada passa a ser $\Theta(n \log n)$, e o algoritmo inteiro também vira $\Theta(n \log n)$.

Verificação em Java (Selection Sort vs. Arrays.sort, tempo real medido)

```
long t0 = System.nanoTime();
selectionSort(a1);
long t1 = System.nanoTime();
Arrays.sort(a2);
long t2 = System.nanoTime();
```

```
n       SelectionSort O(n^2) ms   Arrays.sort O(n log n) ms   ganho (x)
1000    3.5040                     0.7784                     4.5x
5000    13.0737                    3.3963                     3.8x
10000   28.1176                    1.7824                     15.8x
20000   111.5333                   4.3965                     25.4x
```

O ganho de velocidade aumenta conforme n cresce (de 4,5x para 25,4x apenas indo de 1.000 para 20.000 elementos) — é a diferença entre $\Theta(n^2)$ e $\Theta(n \log n)$ se tornando visível na prática, exatamente como a análise assintótica previu. Menos tempo de CPU também significa menos energia consumida: a troca do algoritmo é, ao mesmo tempo, uma melhoria de desempenho e uma decisão de sustentabilidade computacional.